Peer Review: Shegofa Alizada

Mathew Kang

Code Review and Efficiency

The most inefficient code in their homework was in Question 7. While they imported the `rdrobust` package, instead of using it as advised, they used the base-R `lm()` function to run an OLS regression for every bandwidth, as shown:

```
star25.1 <- lm(market_share ~ score + treat + score_treat, data= (ma.rd2 %>%
↪  filter(window1==TRUE)))
star25.2 <- lm(market_share ~ score + treat + score_treat, data= (ma.rd2 %>%
↪  filter(window2==TRUE)))
star25.3 <- lm(market_share ~ score + treat + score_treat, data= (ma.rd2 %>%
↪  filter(window3==TRUE)))
star25.4 <- lm(market_share ~ score + treat + score_treat, data= (ma.rd2 %>%
↪  filter(window4==TRUE)))
star25.5 <- lm(market_share ~ score + treat + score_treat, data= (ma.rd2 %>%
↪  filter(window5==TRUE)))
```

They, then, proceeded to write another 5 lines of code to isolate the coefficients. However, by combining the bandwidths into a vector and running a for loop using the `rdrobust()` function, not only would this have shortened their code, but it would have yielded identical results. Using their same dataframe for the 3.0-star and 3.5-star cutoffs as a reference, this would have optimized their code for better clarity:

```
# Define bandwidths
bandwidths <- c(0.1, 0.12, 0.13, 0.14, 0.15)

# Create an empty dataframe
sens_results <- data.frame()

# Run a for loop for both cutoffs
for(h in bandwidths) {
  fit_275 <- rdrobust(y = ma.rd1$market_share, x = ma.rd1$score, c = 0, h = h, p
↪  = 1, kernel = "uniform", vce = "hc0", masspoints = "off")
  fit_325 <- rdrobust(y = ma.rd2$market_share, x = ma.rd2$score, c = 0, h = h, p
↪  = 1, kernel = "uniform", vce = "hc0", masspoints = "off")
  temp_results <- data.frame(
```

```
    Bandwidth = h,
    Estimate = c(fit_275$Estimate[1], fit_325$Estimate[1]),
    Comparison = c("2.5 vs 3.0 Stars", "3.0 vs 3.5 Stars")
  )

  sens_results <- rbind(sens_results, temp_results)
}
```

While not a perfect solution, this 21-line chunk prepares a whole dataframe of RD estimates to be graphed, while my peer uses 20 lines just to run the regressions and extract the coefficients.

Code Review and Efficiency

For the most part, my peer's code is nearly identical to what I have in my most recent submission. However, there were notable differences, mainly in question 8. In their density continuity plots, their peaks in star ratings before the 2.75-star cutoff were much higher compared to mine. The same goes for the density continuity plots for the 3.25-star cutoff. This is clearly due to my y-axis being shorter than theirs. However, seeing as how I didn't set a limit to the y-axis, I suspect that our estimates may be different rather than simply our visualizations, despite our code being functionally identical.

Replicability and Documentation

While I was able to clone my peer's repository, I was not able to replicate their results.

Diagnosing Replication

The reason I wasn't able to replicate my peer's results is fairly simple: when they imported the raw data, they used the absolute file path instead of the relative file path across all data types (enrollment and contract data, service area data, star ratings, etc.) and years. A close inspection of the file path shows that their abbreviated name is what's preventing me from importing and cleaning the data. For example, when they import penetration data, they use the following path:

```
load_month_pen <- function(m, y) {
    path <-
↪   paste0("/home/salizad/econ470/a0/work/ma-data/ma/penetration/Extracted
↪   Data/State_County_Penetration_MA_",y, "_", m, ".csv")
```

By matching the relative path to the directory tree in the website under "Assignments," I'm able to run the annual data cleaning files. These changes are shown in the annual data cleaning files appended with `.fixed`. An example of the path change is shown below:

```
load_month_pen <- function(m, y) {
    path <- paste0("data/input/ma/penetration/Extracted
 ↪  Data/State_County_Penetration_MA_",y, "_", m, ".csv")
```

Even after adjusting the file paths, I was not able to replicate their results if I tried to run all cells at once. The problem stemmed from this function, where they tried to convert what I assume is `parent_org` to characters:

```
fix_type <- function(df) {
  df$org_parent <- as.character(df$org_parent)
  df
}
ma.2011 <- fix_type(ma.2011)
```

`org_parent` does not exist as a variable. It is also redundant, since `parent_org` is already defined as a character in "functions-1.R" and isn't used in any part of their submission.

Final Thoughts

I think my peer's approach to question 5 was a lot more intuitive than mine. They use the `case_when()` function with a relatively straightforward condition to return the number of plans corresponding to each rounded star rating:

```
round_up_counts <- ma.2010 %>%
  filter(!is.na(raw_rating)) %>%
  filter(!is.na(partc_score)) %>%
  mutate(
    round_ups = case_when(
      raw_rating >= 2.75 & partc_score == 3.0  ~ "Rounded up to 3",
      raw_rating >= 3.25 & partc_score == 3.5  ~ "Rounded up to 3.5",
      raw_rating >= 3.75 & partc_score == 4.0  ~ "Rounded up to 4",
      raw_rating >= 4.25 & partc_score == 4.5  ~ "Rounded up to 4.5",
      raw_rating >= 4.75 & partc_score == 5.0  ~ "Rounded up to 5",
      TRUE ~ NA_character_)
  ) %>%
  filter(!is.na(round_ups)) %>%
  count(round_ups) %>%
  rename('Star Rating' = round_ups, 'Corresponding Number Of Plans' = n)
```

On the other hand, I defined the cutoffs into a tibble and used the `pmap_dfr()` function to map them parallel-row-wise onto my dataset to achieve the same result. It was incredibly difficult trying to understand how `pmap_dfr()` works, but after seeing my peer's work, I realize that there's no use trying to learn it when there's easier alternatives. If I could do my assignment again, I would absolutely update my own code to use the `case_when()` function to avoid this unnecessary headache.